
Prof. Florian Tramèr

Assignment 3

02/05/2026 - 14:49h

You can obtain at most 100 points in this assignment. The grading scheme is the following:

Final assignment grade	Total number of points
6.0	[95, 100]
5.75	[91, 95)
5.5	[86, 91)
5.25	[80, 86)
5.0	[73, 80)
4.75	[65, 73)
4.5	[58, 65)
4.25	[52, 58)
4.0	[45, 52)

Collaboration policy: This assignment is *individual* work, and you are required to submit your own solutions. You can *collaborate* with peers in the form of brainstorming, working together, and cross-checking of solutions. However, *you must explicitly state who you worked with* and, crucially, *you must code your solutions without other people's help*. We require a compiled [Declaration of Originality and AI Assistance](#) as an additional document.

The **deadline** for submitting the assignment is **June 5th 2026, at 23:59 Zurich time**. **Late submissions will not be graded.**

How to submit: You need to submit a zip file on Moodle, with the name

llm_assignment_3_submission-STUDENT_ID.zip,

where STUDENT_ID is your student ID (which is formatted like 00-000-000 and can be found on your student card). **There is an automated script for grading. If your files do not follow the naming convention or the data format, you will get 0 points for the task.**

Formatting details for each file are explained in the description of each exercise. The provided Jupyter/Colab notebooks include code to save the files in the correct format.

Moreover, we provide a [validation script](#) that checks the formatting of your zip archive. Instructions on how to run it are commented out in the script. The script requires Python 3.11 or newer. We recommend running it with [uv](#), but you can just run it in a virtual environment with Python ≥ 3.11 and numpy installed. Be mindful of all warnings raised by the script. Note that **the script does not check the correctness of the solutions**.

All required files listed on the next page should be named exactly as specified.

The zip archive should contain the following files.

- `declaration_originality.pdf`
- `Q1_gens.npy`
- `Q1_guesses.npy`
- `Q2_key.txt` (mandatory if you want any Q2 sub-exercise to be graded)
- `Q2_code.py` OR `Q2_code.ipynb` (mandatory if you want any Q2 sub-exercise to be graded. This is an **exclusive OR**: you are not allowed to submit both!)
- `Q2_pvalue.npy`
- `Q2_bool.npy`
- `Q3_guesses.txt`

Question 1: Watermarking LLM outputs (25 pts)

Overview. In this question, you will implement various approaches for watermarking LLM outputs (see the lecture on LLM Security for a short introduction to watermarking).

Specifically, you will implement three watermarking schemes:

1. A dummy scheme that generates text that never contains any letter ‘e’ (lowercase or uppercase).
2. A red-list scheme that proceeds as follows: when generating a token from the LLM, use the value of the previous token to seed a PRNG, and use this PRNG to randomly split the set of tokens into a “red list” and a “green list”. Then, only output a token from the green list.
3. The above scheme can be too strict. E.g., suppose you just generated the token ‘Barack’, and the token ‘Obama’ ends up in the red list. Then your LLM can never generate the string ‘Barack Obama’. To alleviate this, implement a soft version of the red-list scheme: instead of outright banning the LLM from outputting tokens from the red list, we will simply bias the model against these tokens, by reducing the logit scores of all tokens in the red list by some value γ .

For 20 different text prompts that you are given, you should then generate 100 tokens with the LLM using each of these watermarking schemes.

Once you have the watermarking schemes working, think about how you would *detect* these watermarks.

For this, we will give you 80 pieces of text, that have been generated either with: (1) no watermarking; (2) the dummy “no e” watermark; (3) a red list watermark; (4) a soft red list watermark. You should then figure out which piece of text was produced with which type of watermarking.

Instructions. You will work with the following Google Colab Notebook:

https://colab.research.google.com/drive/1EZ_MLOvmzdL4Uh7e8j22nHYkxoHgxlL2.

We use the GPT-2 LLM. While this model is a bit dated (and thus does not produce text as fluent as contemporary LLMs), it is small enough to run easily on the GPUs you can get for free on Colab!

The Colab Notebook contains all the instructions to get you started. In particular, we provide you with a default configuration for text generation, as well as simple PRNG implementation to convert tokens to red lists.

Hint. If you have difficulty solving the second subquestion, remember that there can exist many sequences of tokens that correspond to the same text.

Submission. For this question, you will submit your solutions as two NumPy arrays.

1. `Q1_gens.npy`, is an array of 60 strings that contains your watermarked generations (3 different watermarking schemes applied to 20 different prompts).
2. `Q1_guesses.npy`, is an array of integers of size 80 that contains your guess of which watermarking scheme was used for each of the 80 provided text pieces (1 = no watermarking; 2 = the dummy “no e” watermark; 3 = a red list watermark; 4 = a soft red list watermark). **(each of the 4 options was used exactly 20 times).**

Question 2: Breaking a LLM watermark (45 pts)

Overview. In this question, you will implement an attack that breaks a watermark by appending a specially constructed suffix to watermarked text. Each of you will get a few different watermarked strings created by prompting Pythia-70m (vocab size > 50000) with a method similar to the one described in Question 1. Your task is to find a universal suffix¹ that, when appended to **any** of these strings, flips the classification of the text to “non-watermarked”.

Figuring out the watermark algorithm from the watermarked strings only is considered a difficult cryptographic problem. For this question, you will also get access to a *API service* that returns whether a given string is watermarked. This will allow you to search for suffixes that can undo the watermark.

Since working with strings would require tokenization, and token merges can get messy, the entire task is designed in the token space. This means you will only deal with lists of token ids. The original watermarked strings, your queries, and your final submission will be arrays of comma-separated integers corresponding to tokens **in the model vocabulary**. All submitted integers must be in the vocabulary range $[0, 50277)$.

Subtasks. This question contains two different subtasks with different watermarking keys. Although they are similar in nature, each of these subtasks has different watermarked texts and a different solution. The main difference between the subtasks is the information you will obtain from the API:

1. **p-value API**, where you get the p-value of the submitted token ids, under the null hypothesis that the text is generated without using the watermarking scheme and each token has an independent probability of 0.5 to be in the red list. A string is considered watermarked if the p-value is smaller than 0.01. Your goal is to produce a universal suffix that, when appended to **any** of the original watermarked strings, yields a p-value of at least 0.01.
2. **boolean API**, is a slightly harder challenge where you will only get a boolean indicating the result of the hypothesis test. The API returns 1 if the p-value of the submitted text under the null hypothesis is smaller than 0.01, and 0 otherwise. The goal is again to produce a suffix that, when appended to **any** of the original watermarked strings, yields a p-value of at least 0.01.

p-value computation. The p -value is determined from the number of unique red consecutive pairs and total consecutive pairs by the following pseudocode:

```
def z_score_binomial(positives: int, total: int, p: float = 0.5) -> float:
    z = (positives - total * p) / (total * p * (1 - p)) ** 0.5
    return z

z_value = z_score_binomial(red_pairs, total_pairs, 0.5)
p_value = 1 - scipy.stats.norm.cdf(-z_value)
```

We skip duplicate pairs of consecutive tokens; see the section on “Evaluating Repetitive Text” in <https://arxiv.org/abs/2301.10226> for details. Both APIs reject requests that are too short or have too few pairs of consecutive tokens.

Watermarking scheme. The watermarking scheme is not identical to the one you’ll implement in Question 1, and might construct the red list differently. We do guarantee that whether some token is red or green is determined only by this token and the token before it. We also guarantee that the boolean API uses the same procedure as the p-value API, **but uses a different watermarking seed**, so there is no information about the red and green lists to be gained from one subtask to another.

¹You will need to submit 2 universal suffixes, one for each API.

Interacting with the API. Every student gets a unique API key that is used to identify when interacting with the API. Internally, this API key is linked with your secret watermarking keys. You can find an interactive API documentation under <https://isl-llms.ethz.ch/docs>. Check the sample code at https://colab.research.google.com/drive/1vav6tX6B9kiWTX_YEcJHY4c4gJekqf7w to get familiar with the API calls from Python. We explain the different endpoints below:

1. GET <https://isl-llms.ethz.ch/api/tokens/>. Returns a list with your assigned watermarked lists of tokens. Each entry in the returned list is a dictionary with two keys. (1) `tokens` contains the watermarked token ids as a string with comma-separated integers. (2) `endpoint` can either be `bool` or `pvalue`. This indicates which watermark was used and which watermarking endpoint must be used for each list of token ids. Tokens with `bool` endpoint should be checked against the boolean API and tokens with `pvalue` endpoint against the p-value API. Return schema:

```
{
  tokens: string,
  endpoint: bool | pvalue
}
```

2. POST https://isl-llms.ethz.ch/api/watermark/get_pvalue. Given a list of token ids, returns the p-value for the watermark under the null hypothesis that the text is generated without using the watermarking scheme. A string is considered watermarked if the p-value is smaller than 0.01. Do not use this endpoint with token sequences labeled `bool`. Return schema:

```
{
  submitted_tokens: string,
  wmark_response: float,
}
```

3. POST https://isl-llms.ethz.ch/api/watermark/get_bool. Given a list of token ids, returns 1 if the p-value is smaller than 0.01, or 0 otherwise. Do not use this endpoint with token sequences labeled `pvalue`. Return schema:

```
{
  submitted_tokens: string,
  wmark_response: 0 | 1,
}
```

Each query can take lists of at most **1000** tokens. Both `api/watermark/` endpoints are rate limited per user. You can only query each of them **3000** times per day, and **12000** times overall. We guarantee both subtasks can be solved using the daily limit of queries. You can check your quota with <https://isl-llms.ethz.ch/check-limits/<api.key>>.

Submission. For this question, you will submit your solutions as one text file, one code file, and two NumPy arrays.

1. `Q2_key.txt` is a file with your API key. We also accept `Q2 key.txt` due to a typo in the notebook.
2. `Q2_code.py` or `Q2_code.ipynb` is a file containing the code you used to solve this problem. You should submit either the code file or the notebook, not both.
3. `Q2_pvalue.npy` is an integer array of size at most 140 that contains the token ids of the suffix that achieves a positive score when appended to all sample sequences, in the watermark scoring setting.
4. `Q2_bool.npy` is an integer array of size at most 140 that contains the token ids of the suffix that achieves a passing verdict when appended to all sample sequences, in the watermark detection setting.

Scoring. Points will be awarded according to the length of the submitted suffixes. Shorter suffixes get more points than longer suffixes. Suffixes longer than 140 tokens get 0 points. The final point in both subtasks may be a bit tedious, depending on your approach.

Table 1: Scoring criteria for p-value API and boolean API tasks.

API Type	Suffix length x	Score
p-value API	≤ 40 tokens	15
	41 tokens	14
	$[42, 50]$ tokens	$8 + (50 - x)/2$
	$[50, 140]$ tokens	$3 + (140 - x)/18$
boolean API	≤ 41 tokens	30
	42 tokens	29
	$[43, 50]$ tokens	$13 + (50 - x)$
	$[50, 140]$ tokens	$3 + (140 - x)/9$

Question 3: Secretly censoring Stable Diffusion with LLM embeddings (30 pts)

Overview. You’ve probably already used image generation AIs like the ones in ChatGPT and Gemini. Stable Diffusion is an open-source image generation model that was state of the art in 2022 and 2023. These systems take in a “prompt” (a string like “A picture of a cat”), and then generate an image that best matches the prompt.

To prevent people from using their system to generate some types of content, in 2022, Stable Diffusion added a *safety filter*, which works as follows:

- When an image x is generated by the system, it is fed through an *image encoder* (based on OpenAI’s CLIP model) that generates an *embedding vector* $z \leftarrow \text{encode}_{\text{img}}(x) \in \mathbb{R}^{1024}$.
- This embedding is then compared to a list of fixed “bad embeddings”. If the cosine similarity between the image’s embedding z and one of these bad embeddings is above some threshold, the image is considered bad and discarded.
- The “bad embeddings” are generated by encoding *text* that corresponds to bad things (e.g., pornography, violence, etc.) This works because the CLIP model used for encoding images is actually a *multimodal* model, than can encode both images and text, with the property that text and images that represent similar things should get encoded to similar embeddings. Concretely, the bad embeddings are computed as $\text{encode}_{\text{text}}(s) \in \mathbb{R}^{1024}$ for some text s .

For some “extra security”, Stable Diffusion did not release the banned text strings s . Instead, they only provide the list of bad embeddings. Your goal is to recover these banned strings.

To make sure you don’t have to work with actual NSFW content, we have created a set of 21 bad embeddings that correspond to fairly benign things we don’t want you to be able to generate images of (one of these bad embeddings is an encoding of the string “The solutions to this assignment”, to make sure you don’t use Stable Diffusion to try and cheat).

Your goal is to recover the pieces of text that we encoded to generate these bad embeddings.

Instructions. You will work with the following Google Colab Notebook:

<https://colab.research.google.com/drive/1bb9I-Y3dApJy9ubFloj6429GstuzdpMh>

The Colab Notebook contains all the instructions to get you started, and to download the data needed for this question.

Submission. Submit your 21 guesses in a file named `Q3_guesses.txt`, with one guess per line. The Colab Notebook contains a code snippet to generate this file for you.

Scoring. The score is 30 points for all guesses correct, 0 points for no guesses correct, and a linear interpolation (rounded down) in between.

A Using Google Colab

Here's a simple workflow to get you started with Colab:

https://scribehov.com/shared/Colab_Workflow_Short_-_n5wpWVS0yizxH1i1Gx0g

Colab gives you access to one GPU “for free”, but depending on your usage and on resource availability, the service may decide not to grant you a GPU. We thus recommend the following:

- While you're familiarizing yourself with the codebase, or thinking about how to solve the problems, disconnect the GPU runtime.
- If you do run out of GPU resources on Colab, there is a backup solution using the Euler cluster (see below).

B Notebooks, Pytorch, NumPy, GPUs

If you have any questions or run into any issues, ask us on Moodle. You can also find good explanations for the things you need to know for the lab (and a lot more) here:

- https://nbviewer.org/github/cgpotts/cs224u/blob/2020-spring/tutorial_numpy.ipynb
- https://nbviewer.org/github/cgpotts/cs224u/blob/2020-spring/tutorial_pytorch.ipynb
- https://nbviewer.org/github/cgpotts/cs224u/blob/2020-spring/tutorial_jupyter_notebooks.ipynb

C Running Jupyter Notebooks on Euler

If you run out of free GPUs on Colab, there is an option to use a GPU running on the Euler cluster as an alternative runtime. The process is a bit more involved, and GPU resources on Euler are limited, so please only do this if you do run out of Colab's free tier.

C.1 Launch a Jupyter Notebook from Euler

1. Connect to Euler (see [instructions](#)).
2. download the [start_jupyter.sh](#) script:

```
$ curl -O https://gist.githubusercontent.com/ftramer/b519b4e3d204189b27f94b058a6c4d20/raw/start_jupyter.sh
```

3. Start a Jupyter notebook server on an Euler node with a GPU:

```
$ sbatch start_jupyter.sh
```

This should print out the message: “Submitted batch job {JOB_ID}”.

4. You can use the command `queue` to check if your job is running. You will also get an email once it starts.

Once it is running, your job will print out information to two local files, [jupyter.out](#) and [jupyter.err](#). The file [jupyter.out](#) should contain something like this:

```
Run the following command on your local machine to enable port forwarding:  
ssh -N -L 8888:{NODE_IP}:8888 {USER}@login.euler.ethz.ch
```

5. Run the command in `jupyter.out` on your **local machine**.

```
$ ssh -N -L 8888:{NODE_IP}:8888 {USER}@login.euler.ethz.ch
```

6. Now open the file `jupyter.err` and copy the URL that is printed at the bottom (this may take a minute to appear). It should look like this:

```
http://127.0.0.1:8888/lab?token=d70a2ae9...
```

7. Open this URL in the browser on your local machine. You should now be able to see the Jupyter notebook interface.

C.2 Shutting down the Jupyter Notebook

Run `queue` on Euler to get the `JOB_ID` of your job. Then run: `scancel JOB_ID`. Finally, kill the SSH tunnel on your local machine by pressing `Ctrl+C`.